

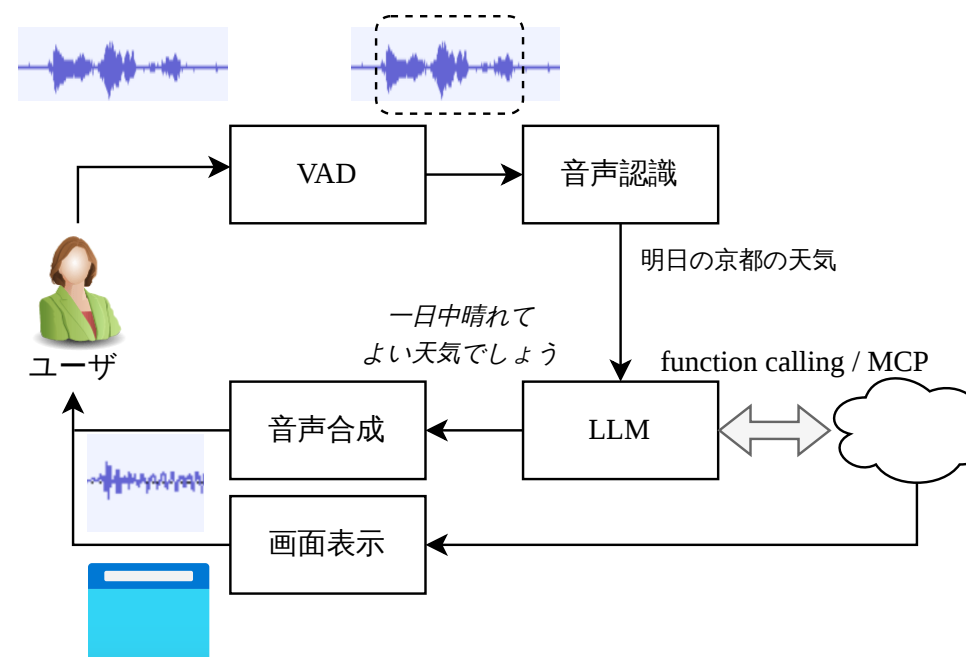
# イラストで学ぶ音声認識 改訂第2版

## 12. 音声対話システム

12.1 モジュール型音声対話システム

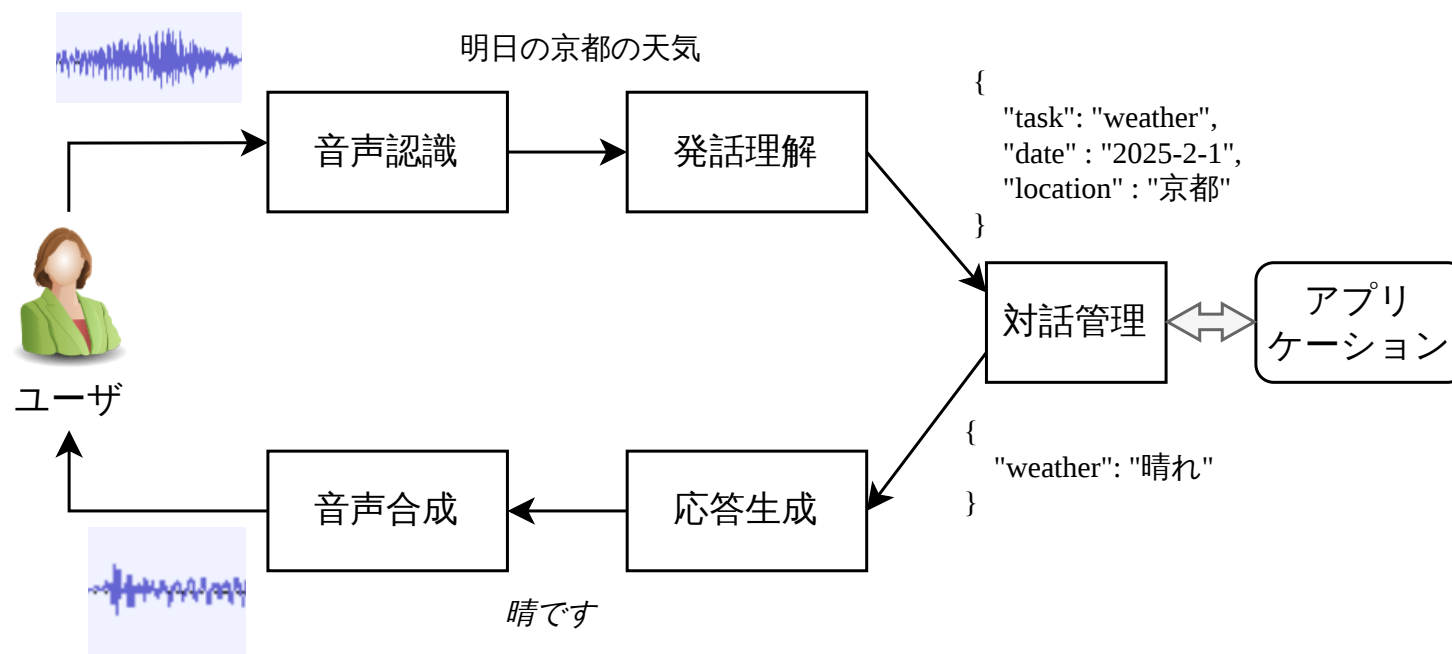
12.2 リアルタイム型音声対話システム

12.3 双方向型音声対話システム



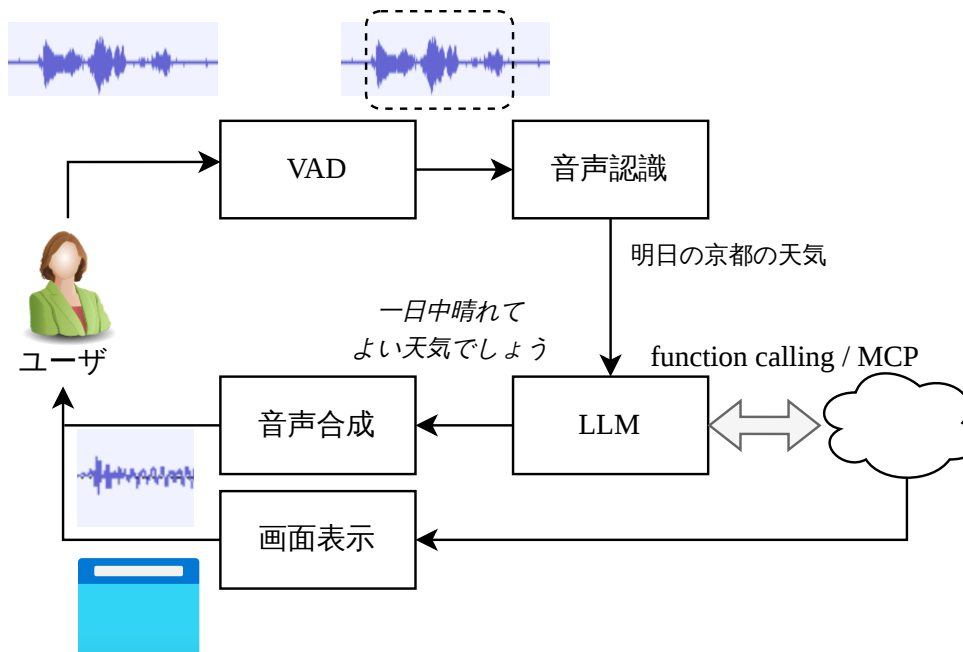
# 12.1 モジュール型音声対話システム

- 従来の音声対話システム
  - 音声入出力に対して発話理解・対話管理・応答生成モジュールを直列に結合
  - それぞれが独立に，小規模なデータで学習していたので性能が低かった



# 12.1 モジュール型音声対話システム

- 大規模言語モデルを用いた音声対話システム
  - 言語を処理する部分をまとめて大規模言語モデル（LLM）で実現
  - VAD（音声区間検出）の高度化やLLMとバックエンドの連携が容易になったことで、より自然で柔軟な対話が可能になった



# 12.1 モジュール型音声対話システム

- 音声対話システムの分類
  - タスク指向対話システム
    - レストラン予約や交通案内など、特定のタスクを遂行することを目的とした対話システム
  - ロールプレイング対話システム
    - 特定の役割を持つキャラクターと対話するシステム
  - 雑談対話システム
    - 特定の目的を持たず、日常的な会話を行うシステム
  - いずれのタイプのシステムも、LLM のプロンプトや外部リソースとの接続方法を工夫することで実現可能

# 12.1 モジュール型音声対話システム

- タスク指向対話システム
  - 対話フローを定義し，LLMに指示することで安定したタスク進行が可能
  - 例：レストラン予約システムの対話フロー

```
flowchart TD
    start([start]) --> ask_date[ask_date]
    ask_date --> ask_number_of_person[ask_number_of_person]
    ask_number_of_person --> ask_cuisine[ask_cuisine[和食, 洋食]]
    ask_cuisine --> confirmation{confirmation}
    confirmation -- yes --> thank([thank])
    confirmation -- no --> modify[modify]
    modify --> confirmation
```

## 12.1 モジュール型音声対話システム

- タスク指向対話システムのためのプロンプトの例

以下の指示に従い，対話フローに応じて [タスクの特定] の対話システムを演じてください。

## 指示

- \* ひとつのシステムからの質問の後，ユーザの入力を待って，次の質問に移ってください
- \* 確認が終了したら，[想定する出力] を JSON で出力してください
- \* [例外事項への対処法の指示]

## 対話フロー

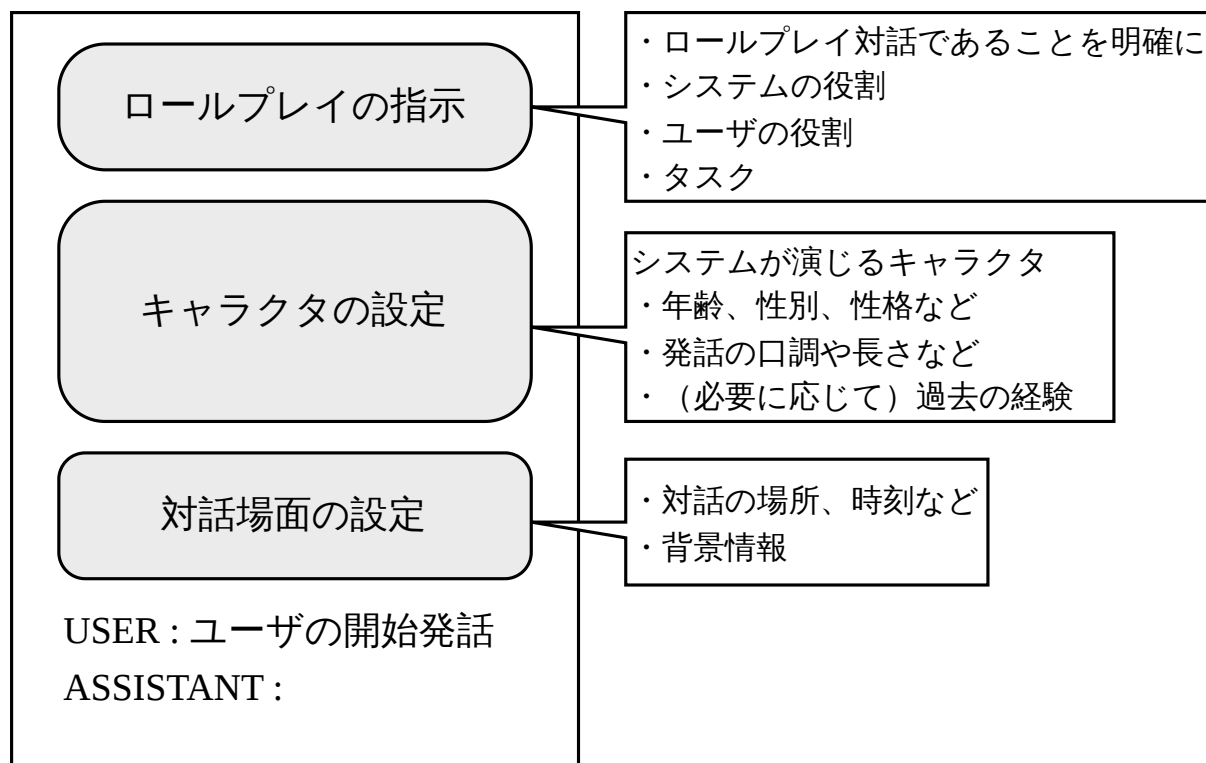
[mermaidで記述]

## 12.1 モジュール型音声対話システム

- ロールプレイング対話システム
  - ユーザとシステムが特定の役割を演じて対話を行うシステム
  - 主として対話による問題解決を行うための練習として，コールセンターのオペレータの訓練や一般の教育目的で用いられる

# 12.1 モジュール型音声対話システム

- ロールプレイング対話システムのプロンプトの構造



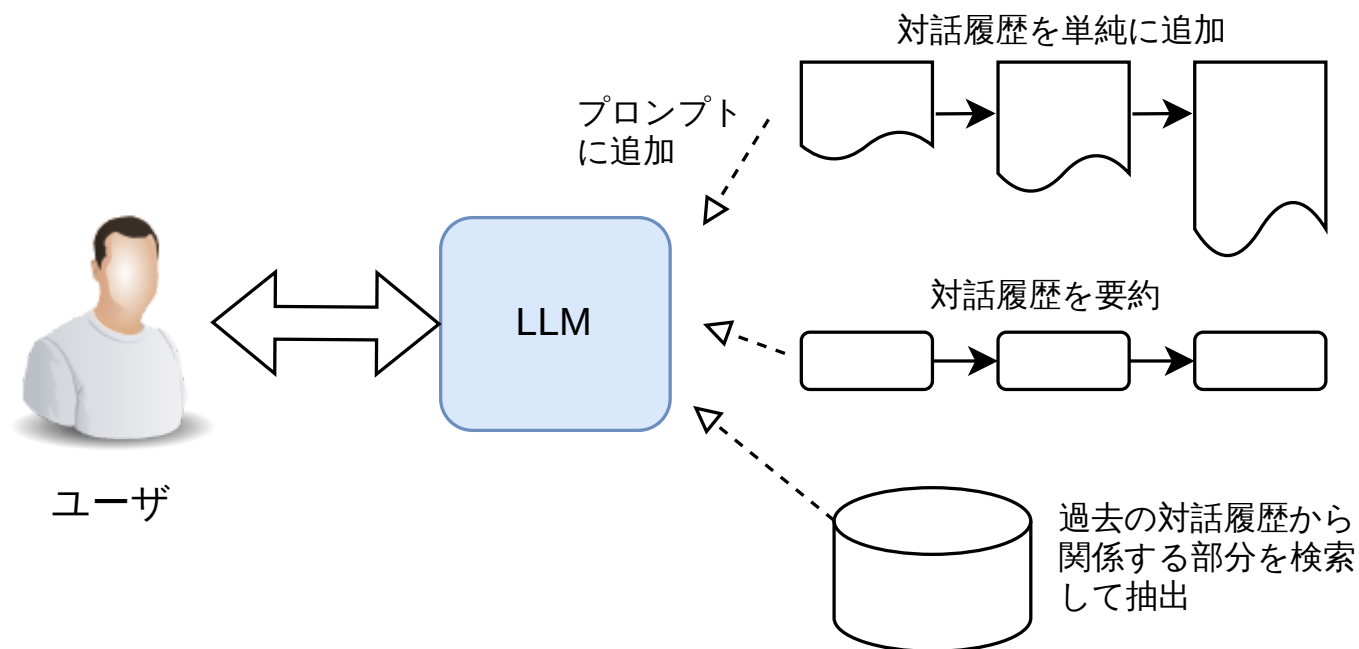


## 12.1 モジュール型音声対話システム

- 雑談対話システム
  - 特定の目的を持たず，日常的な会話を行うシステム
  - LLM を用いることによって，システムはかなり広範囲な話題に対応できるようになった
  - 過去の対話履歴を保存・活用することが重要
    - LangChain や LangGraph などのフレームワークで「メモリ」機能が実装可能

# 12.1 モジュール型音声対話システム

- 雑談対話システムのためのメモリ機能



# 12.1 モジュール型音声対話システム

- 音声入力のコーディング (HTML内部でJavaScriptを使う例)

The diagram illustrates the implementation of a speech recognition system within an HTML document. It features a dark background with white text for the code and white callout boxes with black text for explanations. The code is a complete HTML document with a JavaScript script that uses the WebKit Speech Recognition API. The callouts point to specific parts of the code: '音声認識を利用' points to the `<script>` tag; '日本語を指定' points to `recognition.lang = "ja";`; '認識終了時の処理' points to the `onresult` event listener; and '「音声認識開始」ボタンが押された時の処理' points to the `click` event listener on the `start` button.

```
<!DOCTYPE html>
<html lang="ja">
  <head>
    <meta charset="UTF-8" />
  </head>
  <body>
    <h2>SpeechRecognition APIのデモ</h2>
    <button type="button" class="start">音声認識開始</button>
    <div class="output"></div>
    <script>
      const recognition = new webkitSpeechRecognition();
      recognition.lang = "ja";
      recognition.continuous = true;
      recognition.onresult = ({ results }) => {
        const output = document.querySelector(".output");
        output.textContent = results[0][0].transcript;
      };
      const startButton = document.querySelector(".start");
      startButton.addEventListener("click", () => {
        recognition.start();
      });
    </script>
  </body>
</html>
```

音声認識を利用

日本語を指定

認識終了時の処理

「音声認識開始」ボタンが押された時の処理

## 12.1 モジュール型音声対話システム

- 音声入力のコーディング (Python のライブラリを使う例)

```
import speech_recognition as sr

r = sr.Recognizer()
with sr.Microphone(sample_rate=16000) as source:
    print("なにか話してください")
    audio = r.listen(source)
    print("音声を取得しました")

try:
    print(f"認識結果 {r.recognize_openai(audio)}")
except sr.RequestError as e:
    print(f"失敗！{e}")
```

# 12.1 モジュール型音声対話システム

- 音声出力のコーディング (HTML内部でJavaScriptを使う例)

```
<!DOCTYPE html>
<html lang="ja">
<head>
  <meta charset="UTF-8">
</head>
<body>
  <h1>Web Speech API TTSのデモ</h1>
  <textarea id="text" rows="4" cols="50"
    placeholder="ここにテキストを入力してください"></textarea><br>
  <button onclick="speak()">音声合成開始</button>
  <script>
    function speak() {
      const text = document.getElementById('text').value;
      const utterance = new SpeechSynthesisUtterance(text);
      speechSynthesis.speak(utterance);
    }
  </script>
</body>
</html>
```

「音声合成開始」  
ボタンが押され  
た時の処理

音声合成を利用

音声合成開始

## 12.1 モジュール型音声対話システム

- 音声出力のコーディング (Python のライブラリを使う例)

```
import pyttsx3 as pytts

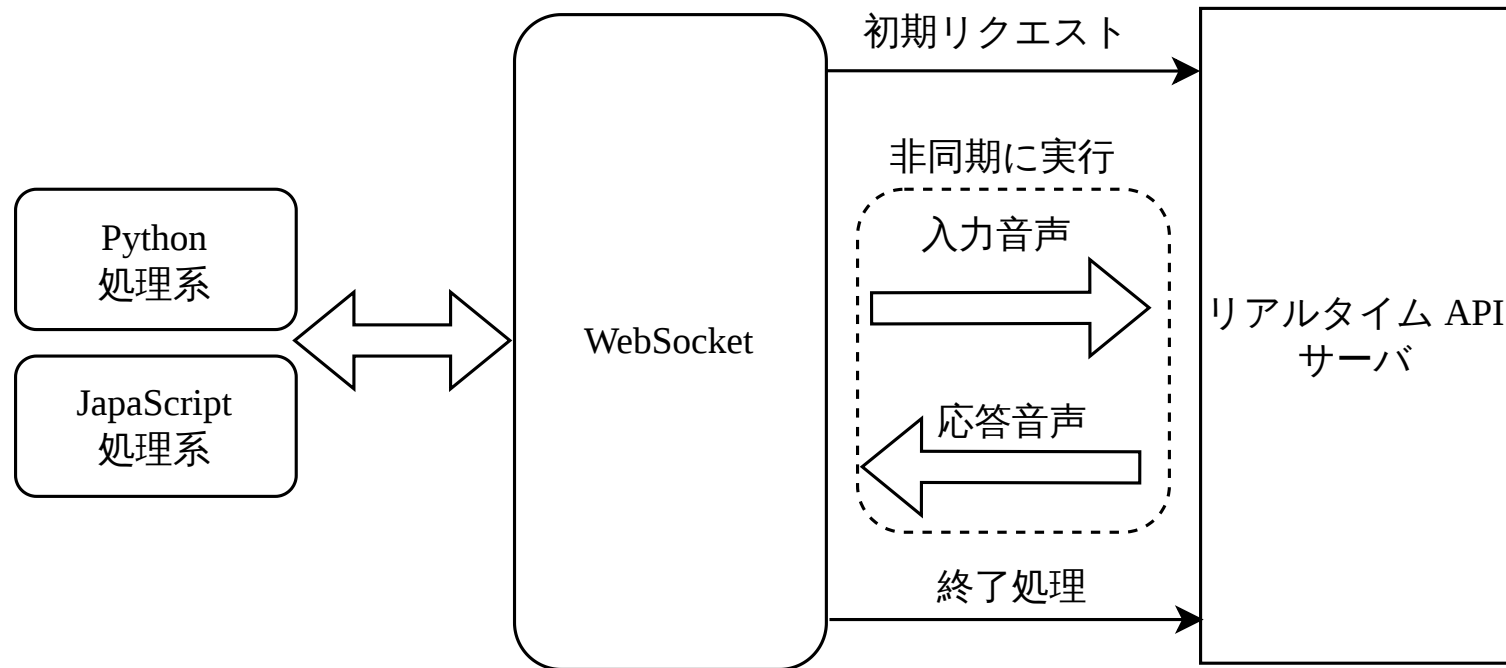
engine = pytts.init()
engine.setProperty('rate', 170)
engine.setProperty('voice', 'ja')
engine.say("このライブラリはOSのTTSエンジンをラップしています。")
engine.runAndWait()
```

## 12.2 リアルタイム型音声対話システム

- リアルタイム型音声対話システムとは
  - ストリーミングで音声入出力を行う
    - ユーザ発話に対して遅延なく応答を返せる
    - システム発話に対する割り込みが生じてても、適切に対応できる
- リアルタイム型音声対話システムの実装
  - WebSocket 等で音声信号の送受信を行い，認識・合成はサーバで行う
  - 認識では音声入力と並行して逐次的にトークン化を行い，LLM に入力する
  - LLMが発話終了と判断した時点で応答文の生成を始める
  - 生成されるトークンを逐次的に音声合成器に送って音声出力を行う

## 12.2 リアルタイム型音声対話システム

- リアルタイム型音声対話システムの構成



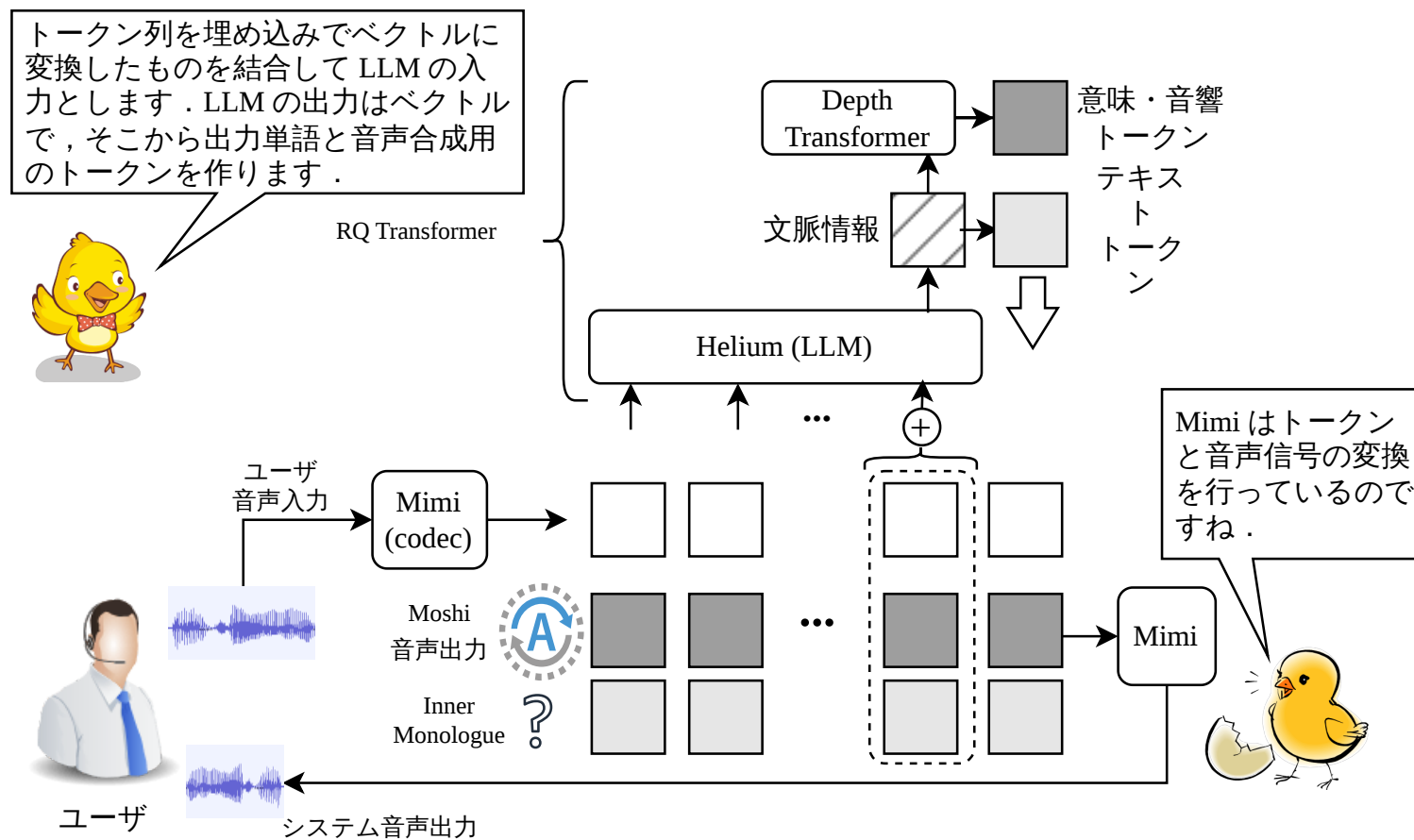


## 12.3 双方向型音声対話システム

- 双方向型音声対話システムとは
  - 一定以上の長さの無音区間を音声終了としたターンの存在を前提としない
  - 音声入出力を大規模言語モデルで同時に処理する
  - 自然な割り込み・オーバーラップ・感情認識や感情表現も可能
- Moshi：音声・テキストのマルチストリームを自己回帰的に処理
  - RVQ（Residual Vector Quantization）と事前学習モデルを用いて音声をトークン化
  - RQ Transformerで文脈を保持しつつ応答テキストと音声を生成
  - システムの応答テキストは Inner Monologue としてトークン化されて自己回帰的な入力となる

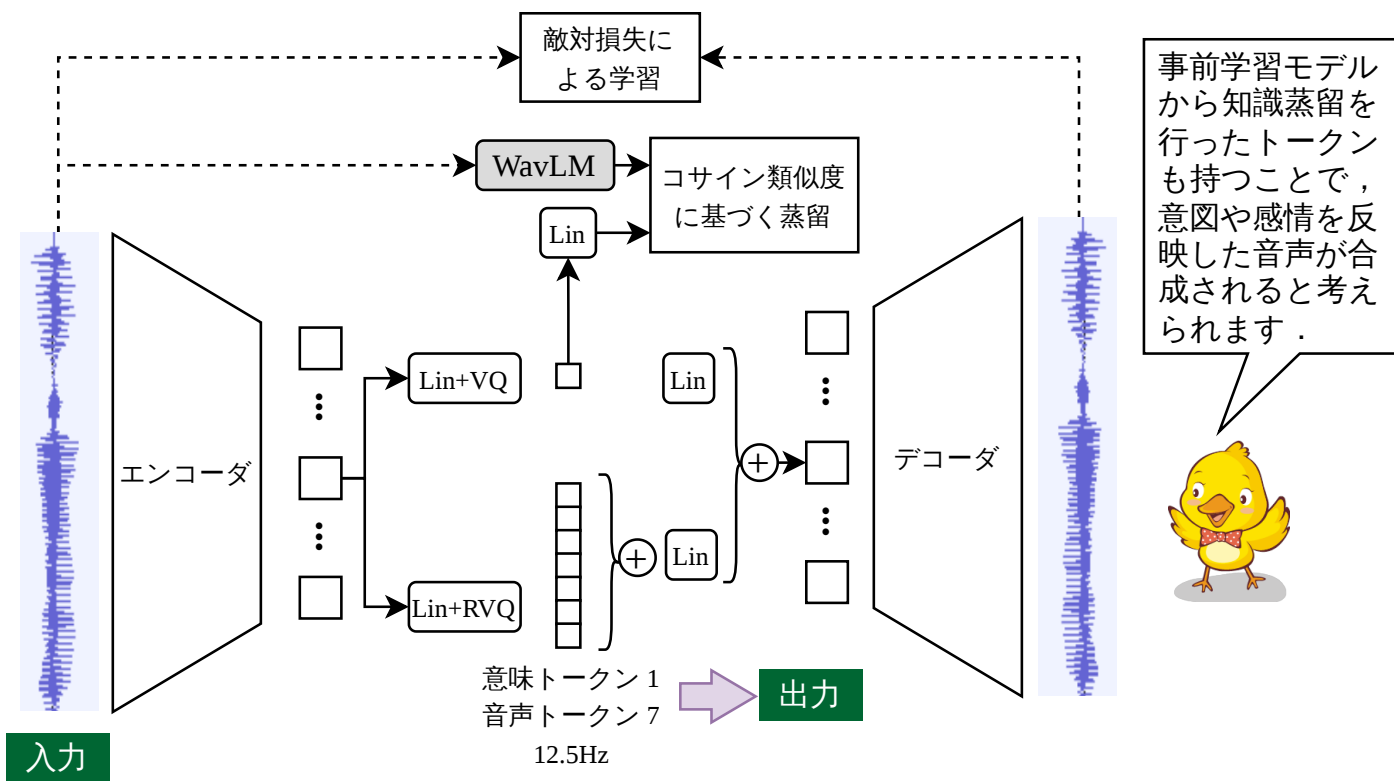
## 12.3 双方向型音声対話システム

- Moshi アーキテクチャ



## 12.3 双方向型音声対話システム

- 音声 codec Mimi
  - 音声信号とトークンとの相互変換を行う



## 12.3 双方向型音声対話システム

- RQ Transformer
  - 音声とテキストの両方の情報を同時に処理し，応答を生成する

